
大数据与人工智能
自动化学院

CHAPTER03.04 模型创建 MODULE、模型容器CONTAINERS 及ALEXNET网络搭建

3.4-1导言

- 通过前面的Pytorch数据读取机制(DataLoader)与图像预处理模块(transforms)，已经整理完了Pytorch的数据模块的一些内容
- 现在正式进入模型模块的学习，进行模型的创建和模型容器的知识学习。
    - 首先学习模型的创建步骤和nn.Module的相关细节
    - 然后学习搭建模型的容器Containers，这里面包括nn.Sequential, nn.ModuleList, nn.ModuleDict， 它们各自有各自的特点和应用场景。
    - 最后我们分析一个经典的网络AlexNet。

本节大纲如下

- Pytorch模型的建立
- Pytorch的容器
- AlexNet网络的构建
- 总结回顾

3.4-2. PYTORCH模型的创建

在学习Pytorch的模型创建之前，我们依然是回顾一下模型创建到底是以什么样的逻辑存在的

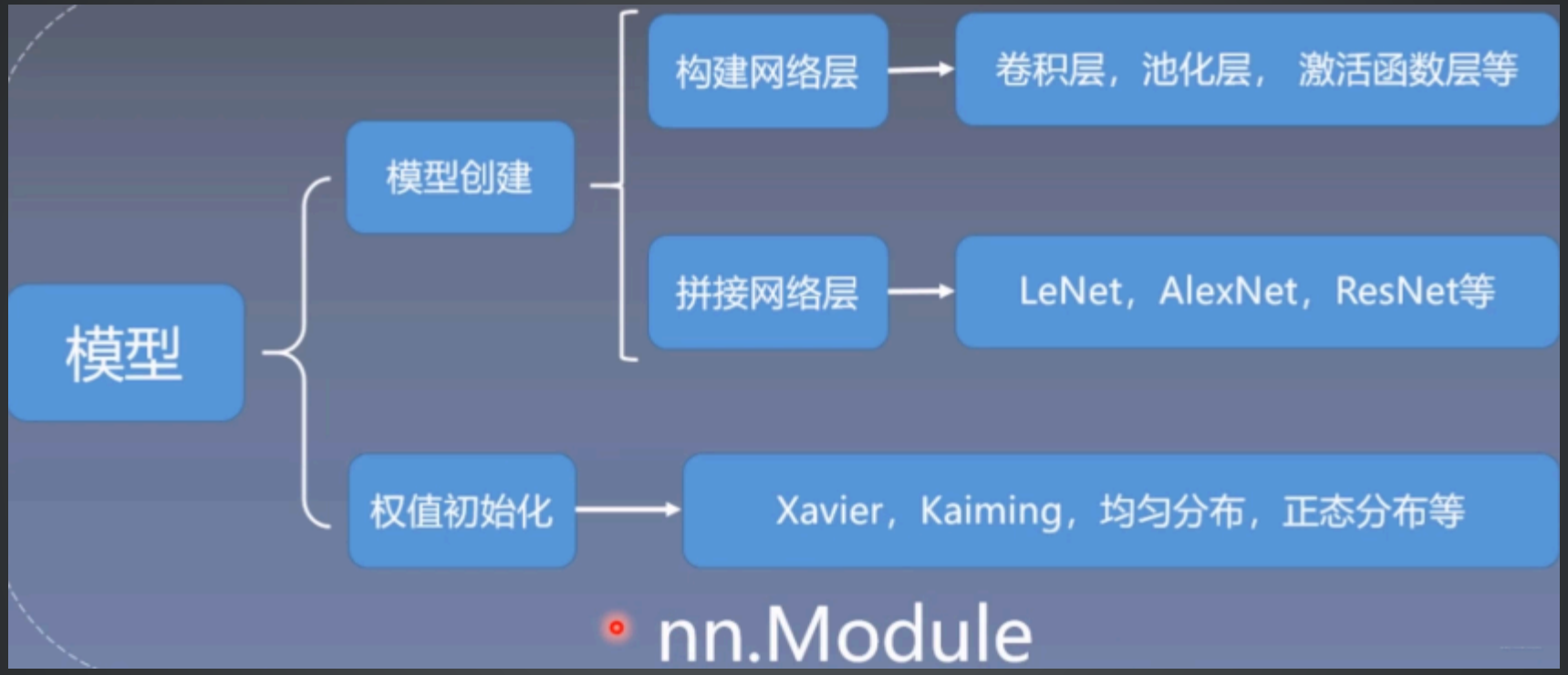
我们已经整理了机器模型学习的五大模块，分别是

数据，**模型**，损失函数，优化器，迭代训练：



这里的模型创建是模型模块的一个分支，和数据模块一样

我们先看一下模型模块的具体内容：

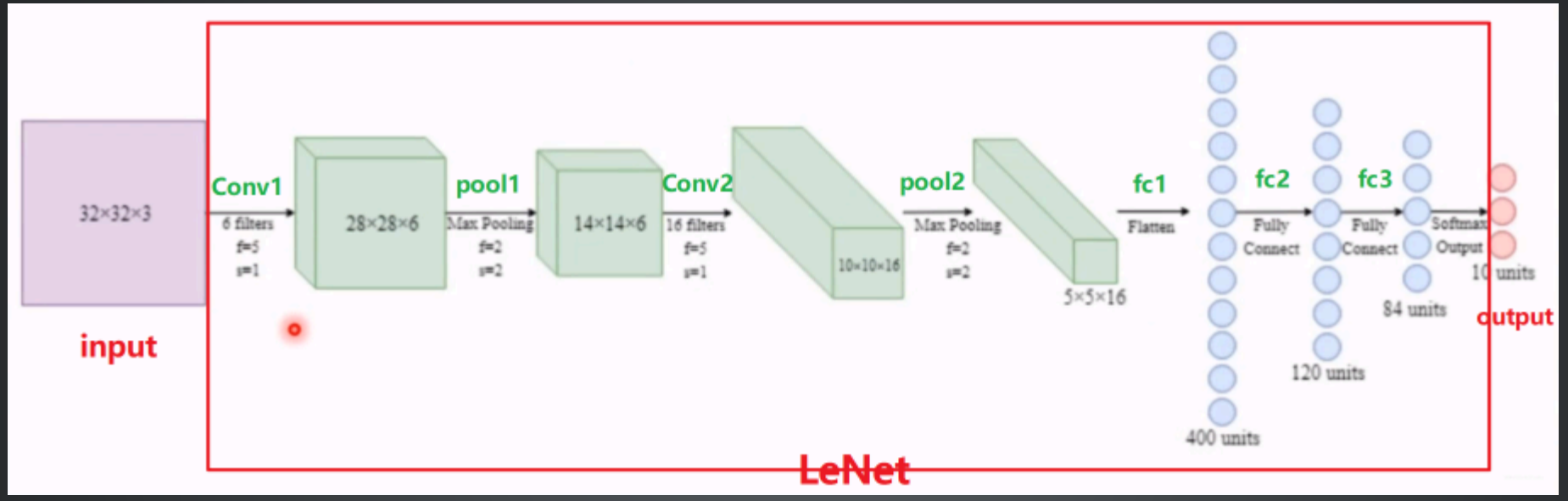


了解了上面这些框架，有利于把知识进行整合起来

到底学习的内容属于哪一块我们从上面的模型创建开始，学习网络层的构建和拼接。

3.4-2.1模型的创建步骤

在模型创建步骤之前，我们先来进行一个分析. 下面是我们在人民币二分类任务过程中用到的LeNet模型



LeNet的模型计算图

- 由边和节点组成，节点就是表示每个数据， 而边就是数据之间的运算
- LeNet是一个很大的网络，接收输入，然后经过运算得到输出
- 在LeNet的内部，又分为很多个子网络层进行拼接组成，这些子网络层之间连接配合，最终完成我们想要的运算。

所以通过上面的分析，我们可以得到构建我们模型的两大要素：

1. 构建子模块（比如LeNet里面的卷积层，池化层，全连接层）
2. 拼接子模块（我们把子模块按照一定的顺序，逻辑进行拼接起来得到最终的LeNet模型）

下面还是以纸币二分类的任务去看看如何进行LeNet模型的构建，依然是使用代码调试：

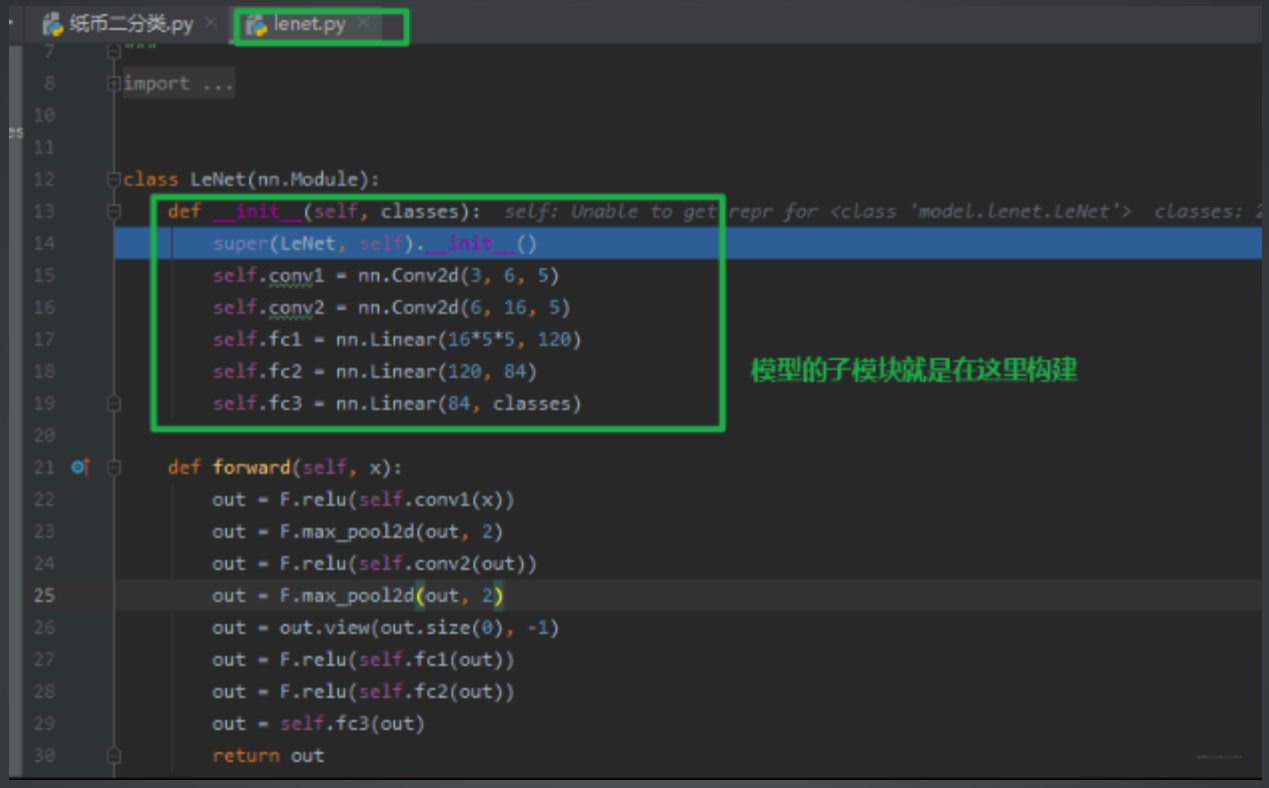
bigAI-03-03-20241118.py | lenet.py | launch.json

01_PyTorch_Practice > bigAI-03-03-20241118.py > ...

```
59  valid_data = RMBDataset(data_dir=valid_dir, transform=valid_transform)
60
61  # 构建DataLoder
62  train_loader = DataLoader(dataset=train_data, batch_size=BATCH_SIZE, shuffle=True)
63  valid_loader = DataLoader(dataset=valid_data, batch_size=BATCH_SIZE)
64
65  # ============================ step 2/5 模型 ============================
66
67  net = LeNet(classes=2)
68  net.initialize_weights()
69
70  # ============================ step 3/5 损失函数 ============================
71  criterion = nn.CrossEntropyLoss()                                   # 选择损失函数
72
```

上一次是数据模块部分，这一次，我们进入模型模块的学习

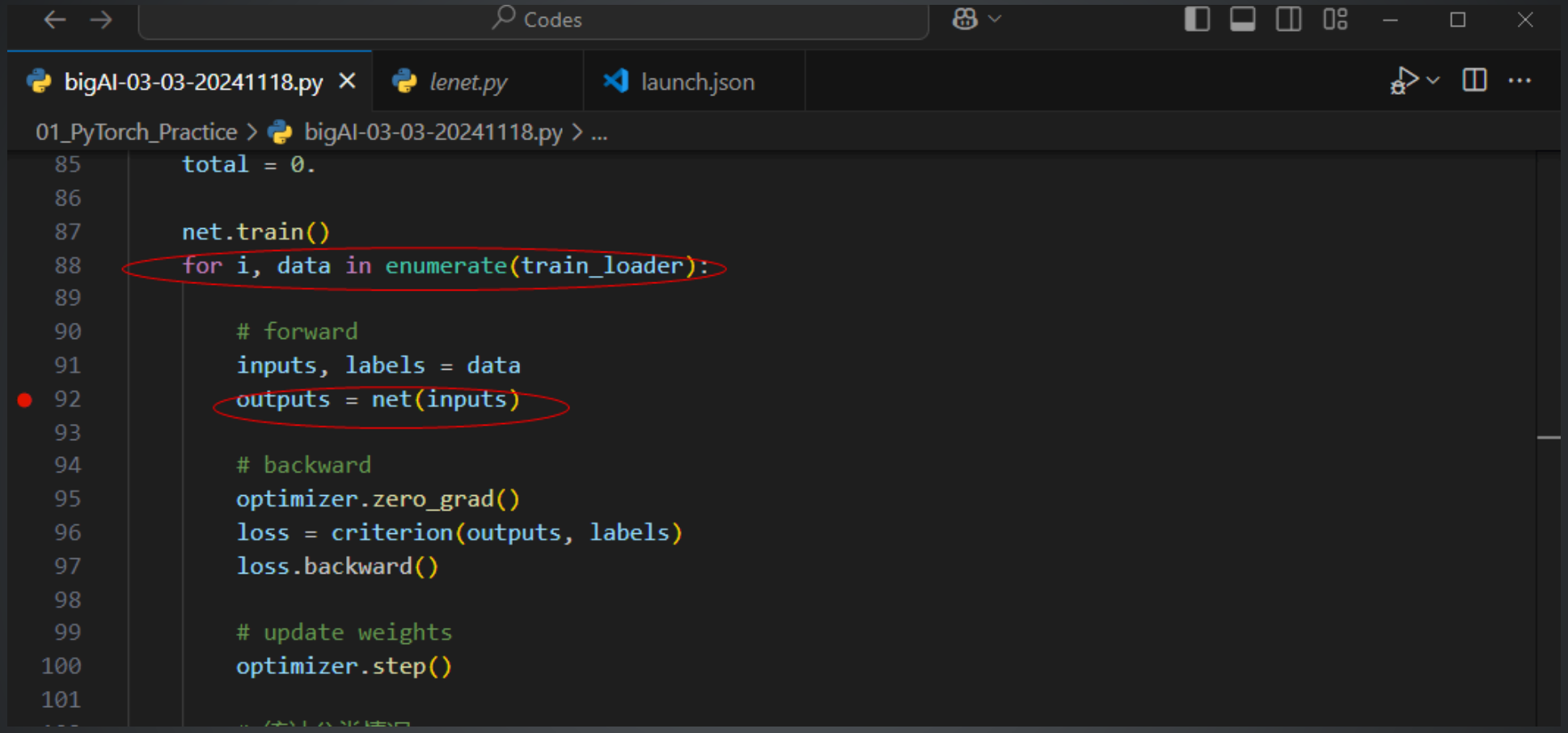
- 这里可以看到，使用的模型是LeNet
- 我们模型这一行打上断点，然后进行debug调试，看看这个LeNet是怎么搭建起来的：



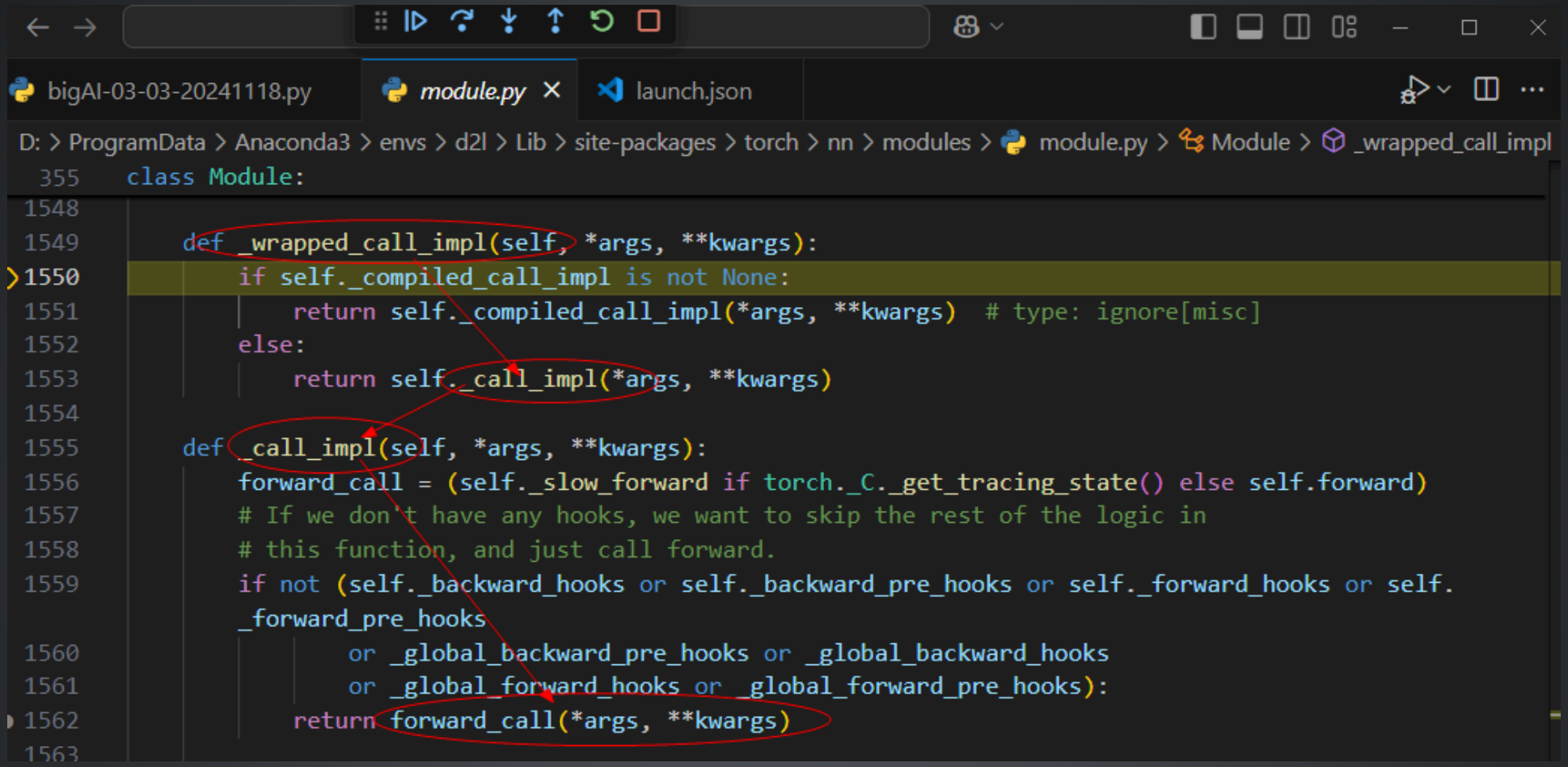
- 程序运行到断点，点击步入，就进入了lenet.py文件
- 在这里面有个**LeNet类，继承了nn.Module**。
- 并且我们发现在**它的__init__方法里面实现了各个子模块的构建**。
- 所以构建模型的第一个要素 - 子模块的构建就是在这里。

下面的一个问题，就是我们知道了子模块是定义在LeNet模型`__init__`方法里面，那么这些子模块的拼接是在哪里定义呢?

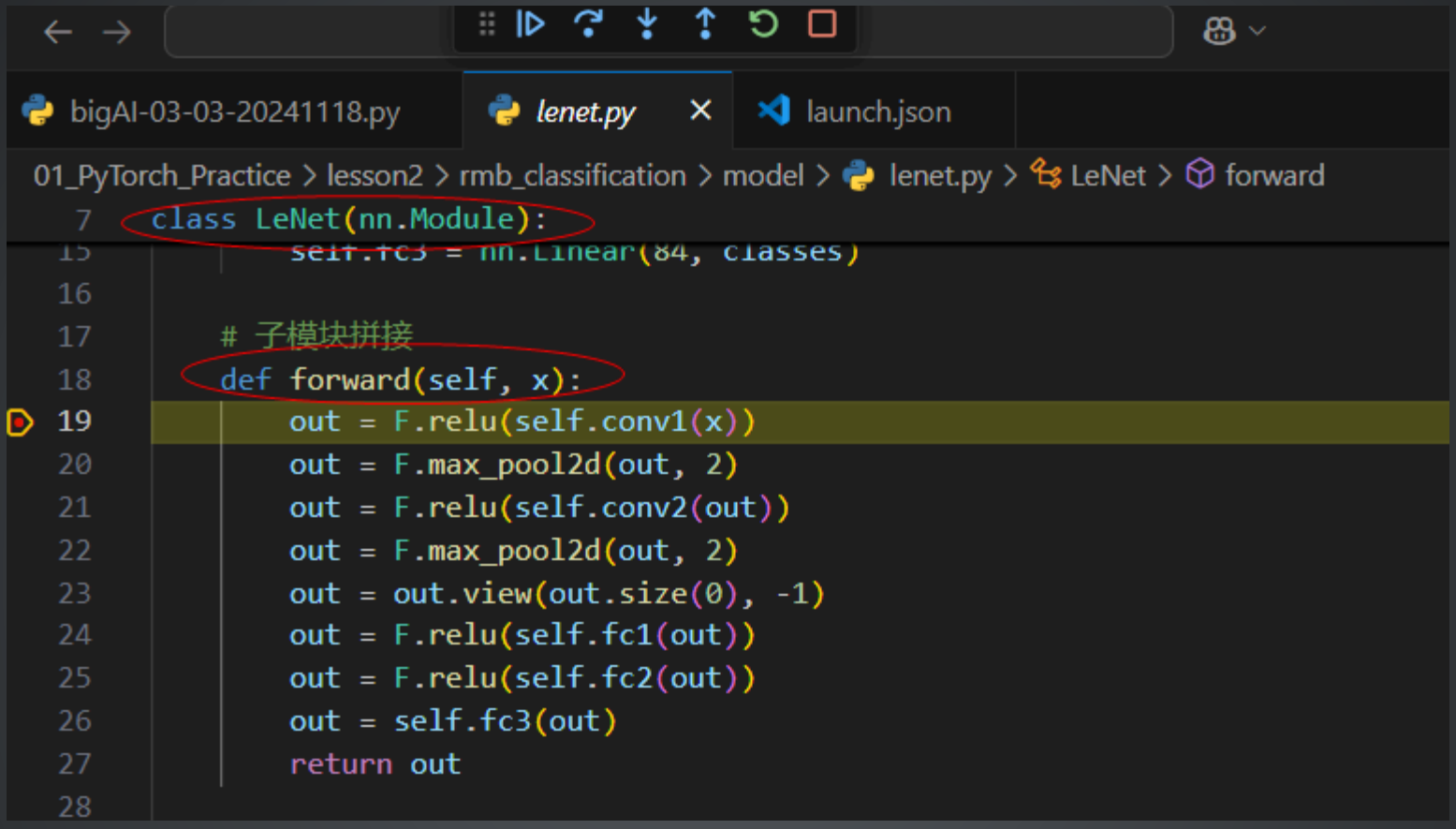
- 其实是在forward里面（后面训练时候，for循环步进才进入）
- 只要程序用到了模型，它必然会知道模型是怎么拼接的
- 所以我们就从模型的训练那进行调试，看看是不是真的调用了forward方法进行模型拼接。



- 主程序的模型训练部分，我们在outputs=net(inputs)打上断点，因为这里开始是模型的训练部分， 而这一行代码正是前向传播，
- 我们进行步入，看看这个函数的运行原理：



- 我们发现进入了module.py里面的一个_wrapped_call_impl函数， 因为我们的LeNet是继承于Module的。
- 然后几次内部调用，step in进入forward_call()
- 我们把鼠标放在这一行，然后运行到这里，再步入，就会发现果真是调用了LeNet的forward方法：



所以，我们基于这个例子，就基本上理清楚了上面构建模型的两个要素：

1. 构建子模块， 这个是在自己建立的模型（继承nn.Module）的__init__()方法
2. 拼接子模块， 这个是在模型的forward()方法中

提问

关于 Pytorch 模型的创建，以下说法正确的是：

A. 模型的构建主要包括子模块的构建和拼接子模块。
B. 子模块的构建通常在 `forward()` 方法中完成。
C. 拼接子模块通常在 `__init__()` 方法中完成。
D. 所有模型都必须继承 `nn.Parameter`。

答案： A
解析：

- Pytorch 模型的创建分为两部分：**子模块的构建**和**拼接子模块**。子模块的构建通常在 `__init__()` 方法中完成，而拼接子模块是在 `forward()` 方法中完成的。选项 B 和 C 错误。
- 模型需要继承的是 `nn.Module`，而不是 `nn.Parameter`，选项 D 错误。

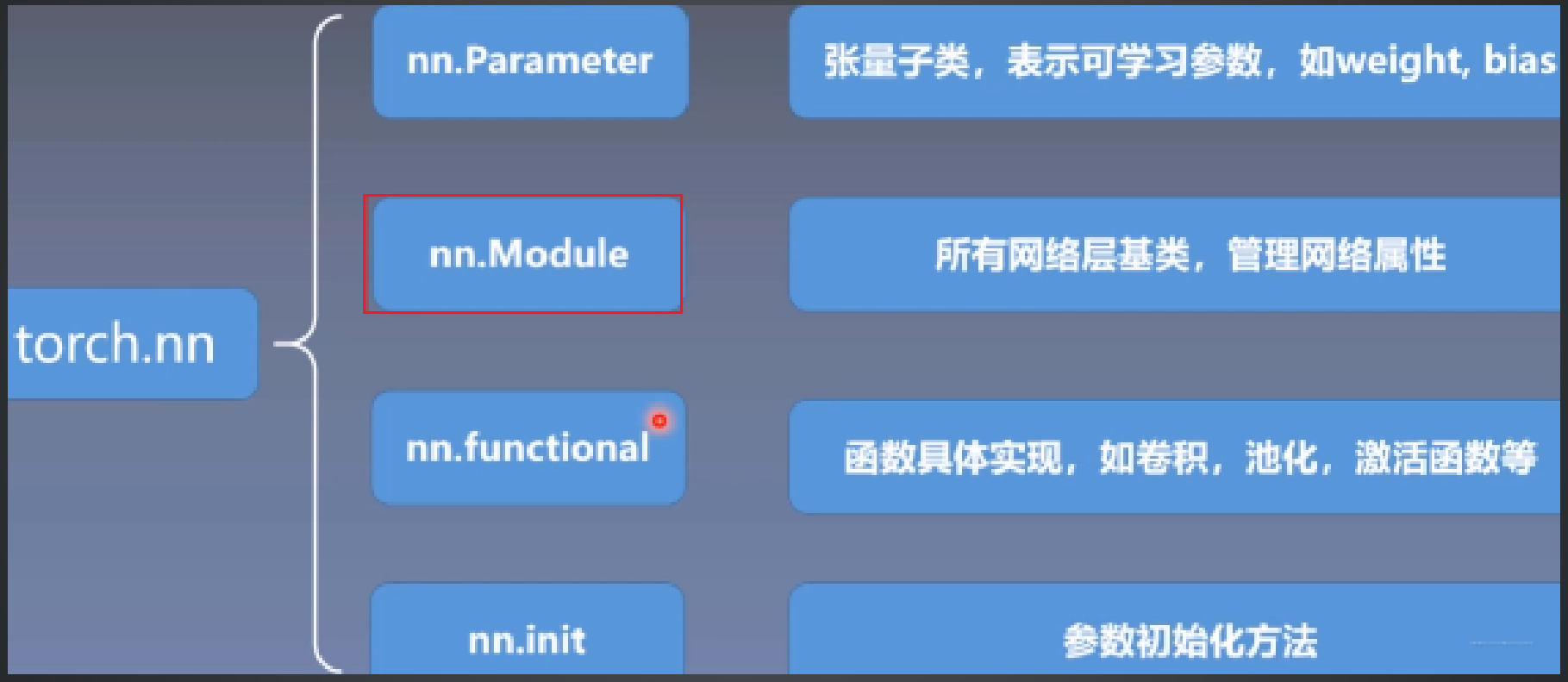
在模型的概念当中，我们有一个非常重要的模块叫做nn.Module

我们所有的模型，所有的网络层都是继承于这个类。

所以我们非常有必要了解nn.Module这个类。

3.4-2.2 NN.MODULE类

我们先介绍与nn.Module其相关的模块， 建立一个框架，把握宏观。



- `torch.nn`: 这是Pytorch的神经网络模块， 这里的Module就是它的子模块之一
- 另外还有几个与Module并列的子模块, 这些子模块协同工作，各司其职。

3.4-2.2.1 NN.PARAMETER

首先是nn.Parameter

- 在Pytorch中，模型的参数需要被优化器训练
- 因此，通常要设置参数为 requires_grad = True 的张量。
- 同时，在一个模型中，往往有许多的参数，要手动管理这些参数并不是一件容易的事情。
- Pytorch一般将参数用nn.Parameter来表示，并且用nn.Module来管理其结构下的所有参数。

```
## nn.Parameter 具有 requires_grad = True 属性
w = nn.Parameter(torch.randn(2,2))
print(w)    # tensor([[ 0.3544, -1.1643],[ 1.2302,  1.3
print(w.requires_grad)    # True


## nn.ParameterList 可以将多个nn.Parameter组成一个列表
params_list = nn.ParameterList([nn.Parameter(torch.ran
print(params_list)
print(params_list[0].requires_grad)


## nn.ParameterDict 可以将多个nn.Parameter组成一个字典
```

我们可以用Module把这些参数管理起来：

```
# module.parameters()返回一个生成器，包括其结构下的所有param

module = nn.Module()
module.w = w
module.params_list = params_list
module.params_dict = params_dict

num_param = 0
for param in module.parameters():
    print(param,"\n")
    num_param = num_param + 1
print("number of Parameters =",num_param)
```

实践当中，一般通过继承nn.Module来构建模块类，并将所有含有需要学习的参数的部分放在构造函数中。

```
#以下范例为Pytorch中nn.Linear的源码的简化版本
#可以看到它将需要学习的参数放在了__init__构造函数中，并在forwa

class Linear(nn.Module):
    __constants__ = ['in_features', 'out_features']
    #__constants__ 是 PyTorch 中的一个特殊属性，用于指定哪些

    def __init__(self, in_features, out_features, bias=
        super(Linear, self).__init__()
        self.in_features = in_features
        self.out_features = out_features

        # 定义了权重矩阵 WW，大小为 [out features, in feat
```

3.4-2.2.2 NN.FUNCTIONAL

nn.functional(一般引入后改名为F，即import torch.nn.function as F)

F有各种功能组件的函数实现。 比如：

- 激活函数系列(F.relu, F.sigmoid, F.tanh, F.softmax)
- 模型层系列(F.linear, F.conv2d, F.max_pool2d, F.dropout2d, F.embedding)
- 损失函数系列(F.binary_cross_entropy, F.mse_loss, F.cross_entropy)

为了便于对参数进行管理， 一般通过继承nn.Module转换为类的实现形式

Pytorch已经将它们直接封装在nn模块下：

- 激活函数变成(nn.ReLu， nn.Sigmoid， nn.Tanh， nn.Softmax)
- 模型层(nn.Linear， nn.Conv2d， nn.MaxPool2d， nn.Embedding)
- 损失函数(nn.BCELoss， nn.MSELoss， nn.CrossEntorpyLoss)

所以我们表面上**用nn建立的这些激活函数， 层， 损失函数， 背后都在functional里面具体实现**。

- nn.Module这个模块确实非常强大
- 除了可以管理其引用的各种参数，
- 还可以管理其引用的子模块。

我们下面就来揭开nn.Module的神秘面纱。

3.4-2.2.3 NN.MODULE

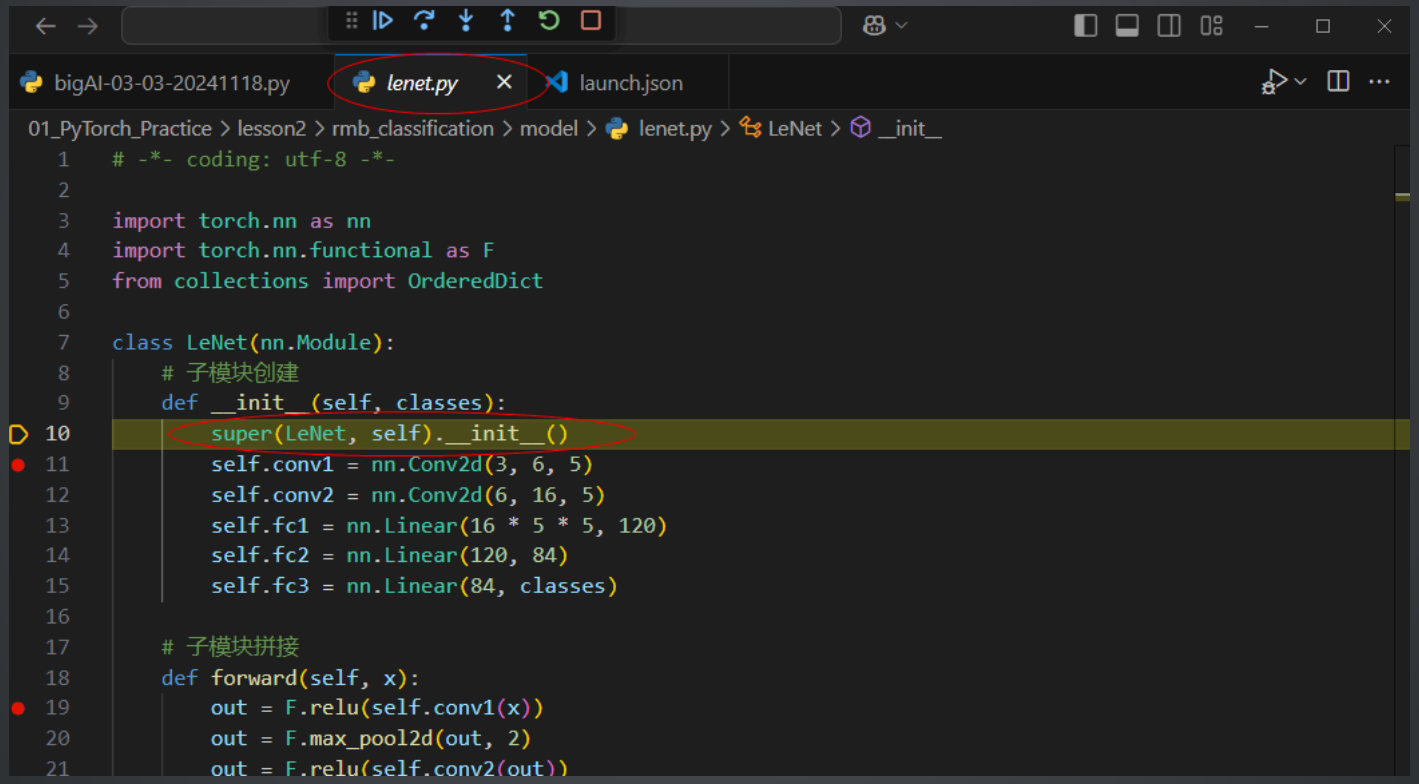
nn.Module这个模块是重点， 这里面是所有网络层的基类，管理有关网络的属性。

在nn.Module中，有8个重要的属性， 用于管理整个模型，他们都是以有序字典的形式存在着：

```
self._parameters = OrderedDict()
self._buffers = OrderedDict()
self._backward_hooks = OrderedDict()
self._forward_hooks = OrderedDict()
self._forward_pre_hooks = OrderedDict()
self._state_dict_hooks = OrderedDict()
self._load_state_dict_pre_hooks = OrderedDict()
self._modules = OrderedDict()
```

- _parameters： 存储管理属于nn.Parameter类的属性，例如权值，偏置这些参数
- _modules: 存储管理nn.Module类， 比如LeNet中，会构建子模块，卷积层，池化层，就会存储在_modules中
- _buffers: 存储管理缓冲属性， 如BN层中的running_mean， std等都会存在这里面
- ***_hooks: 存储管理钩子函数（5个与hooks有关的字典，这个先不用管）

- nn.Module中的属性的重点是： _parameters 以及 _modules
- 下面通过自编代码的lenet.py里面的LeNet模型代码来观察nn.Module的创建以及它对属性管理的一个机制。
- 这里依然开启调试机制， 先在net = LeNet(classes=2)前打上断点，然后debug步入到LeNet。



- 可以看到LeNet是继承于nn.Module的，所以LeNet也是一个Module
- 我们看看__init__方法中的第一行，是实现了父类函数调用的功能，在这里也就是调用了nn.Module的初始化函数。

我们进入这一个nn.Module的__init__

bigAI-03-03-20241118.py | module.py | launch.json

D: > ProgramData > Anaconda3 > envs > d2l > Lib > site-packages > torch > nn > modules > module.py > Module > __init_

```
355   class Module:
429       def __init__(self, *args, **kwargs) -> None:
445           handling for parameters, submodules, and buffers but simply calls into
446           super().__setattr__ for all other attributes.
447           """
448           super().__setattr__('training', True)
449           super().__setattr__('_parameters', OrderedDict())
450           super().__setattr__('_buffers', OrderedDict())
451           super().__setattr__('_non_persistent_buffers_set', set())
452           super().__setattr__('_backward_pre_hooks', OrderedDict())
453           super().__setattr__('_backward_hooks', OrderedDict())
454           super().__setattr__('_is_full_backward_hook', None)
455           super().__setattr__('_forward_hooks', OrderedDict())
456           super().__setattr__('_forward_hooks_with_kwargs', OrderedDict())
457           super().__setattr__('_forward_hooks_always_called', OrderedDict())
458           super().__setattr__('_forward_pre_hooks', OrderedDict())
459           super().__setattr__('_forward_pre_hooks_with_kwargs', OrderedDict())
460           super().__setattr__('_state_dict_hooks', OrderedDict())
461           super().__setattr__('_state_dict_pre_hooks', OrderedDict())
462           super().__setattr__('_load_state_dict_pre_hooks', OrderedDict())
463           super().__setattr__('_load_state_dict_post_hooks', OrderedDict())
464           super().__setattr__('_modules', OrderedDict())
```

- 从Module类里面的初始化方法中，看到了会实现了多个有序字典的一个初始化操作
- 也就是LeNet模型就有了8个有序字典属性，去帮助我们存储后面的变量和参数。

我们跳回去lenet.py， 继续往下运行第二行代码，建立第一个子模块卷积层，

bigAI-03-03-20241118.py | lenet.py | launch.json

01_PyTorch_Practice > lesson2 > rmb_classification > model > lenet.py > LeNet > __init__

```
from collections import OrderedDict

class LeNet(nn.Module):
    # 子模块创建
    def __init__(self, classes):
        super(LeNet, self).__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, classes)

    # 子模块拼接
    def forward(self, x):
        out = F.relu(self.conv1(x))
        out = F.max_pool2d(out, 2)
```

- 使用step in，进入到nn.Conv2d这个卷积层
- 我们会发现class Conv2d(_ConvNd):也就是这个类是继承于_ConvNd的
- 在Conv2d的初始化方法中，依然是先调用父类的初始化方法， 我们进入这个类

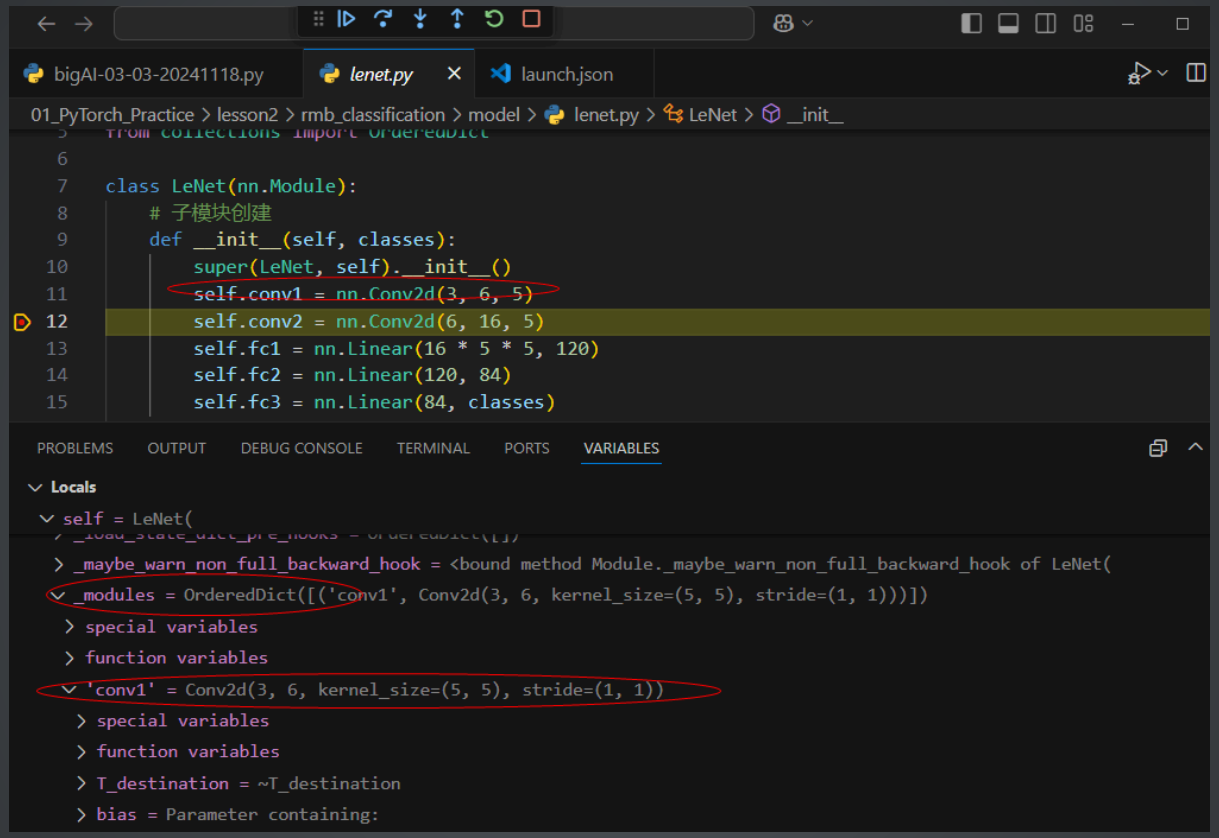
bigAI-03-03-20241118.py | conv.py | launch.json

D: > ProgramData > Anaconda3 > envs > d2l > Lib > site-packages > torch > nn > modules > conv.py > Conv2d >

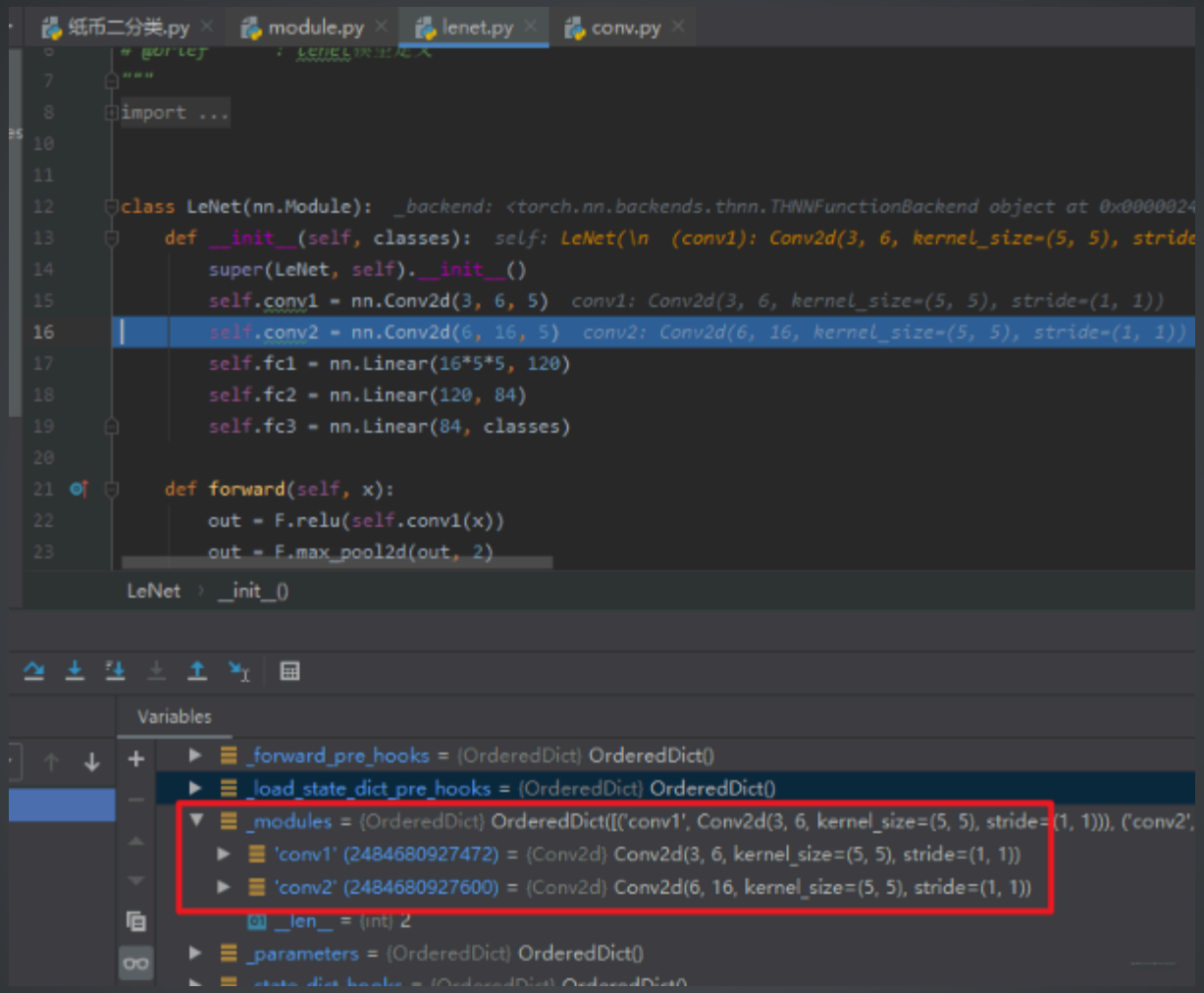
```
311  class Conv2d(_ConvNd):
426      def __init__(
427          self,
428          in_channels: int,
429          out_channels: int,
430          kernel_size: _size_2_t,
431          stride: _size_2_t = 1,
432          padding: Union[str, _size_2_t] = 0,
433          dilation: _size_2_t = 1,
434          groups: int = 1,
435          bias: bool = True,
436          padding_mode: str = 'zeros',  # TODO: refine this type
437          device=None,
438          dtype=None
439      ) -> None:
440          factory_kwargs = {'device': device, 'dtype': dtype}
441          kernel_size_ = _pair(kernel_size)
442          stride_ = _pair(stride)
443          padding_ = padding if isinstance(padding, str) else _pair(padding)
444          dilation_ = _pair(dilation)
445          super().__init__(
446              in_channels, out_channels, kernel_size_, stride_, padding_, dilation_,
447              False, _pair(0), groups, bias, padding_mode, **factory_kwargs)
```

- 我们发现_ConvNd也是继承Module这个类的，并且初始化方法中也是用了super调用了父类的初始化方法。
- 所以这就说明**Conv2d这个子模块是一个Module，并且也有那些有序字典的属性。**

然后我们再跳回去，准备运行第三行代码：



- 前面第二行代码运行完了，也就是我们在LeNet中建立了第一个子模块Conv2d
- 那么我们可以看到LeNet的_modules这个有序字典中，就记录了这个子模块的信息。
- 通过上面的调试，我们就看清楚了LeNet这个Module实现了一个子网络层的构建
- 并且把它存储到了_modules这个字典中进行管理。



这样一步一步的运行下去，就可以把整个网络构建起来。

nn.Module构建属性的机制

- 我们是先有一个大的Module继承nn.Module这个基类， 比如上面的LeNet，
- 然后这个大的Module里面又可以有很多的子模块，这些子模块同样也是继承于nn.Module，
- 在这些Module的__init__方法中，会先通过调用父类的初始化方法进行8个属性的一个初始化。
- 然后在构建每个子模块的时候，其实分为两步，
    - 第一步是初始化，
    - 然后再进行赋值给相应的成员。

这样一个个的构建子模块，最终把整个大的Module构建完毕。

对nn.Module进行总结：

- 一个module可以包含多个子module（LeNet包含卷积层，池化层，全连接层）
- 一个module也需要实现网络的运算， 必须实现forward()函数（从计算图的角度去理解）
- 每个module都有若干字典管理它的属性（最常用的就是_parameters，_modules）

- 一般情况下，我们都很少直接使用 nn.Parameter来定义参数构建模型
- 而是通过拼装一些常用的模型层来构造模型。
- 这些模型层也是继承自nn.Module的对象,本身也包括参数，属于我们要定义的模块的子模块。

nn.Module提供了一些方法可以管理这些子模块。

- children() 方法: 返回生成器，包括模块下的所有子模块。
- named_children()方法：返回一个生成器，包括模块下的所有子模块，以及它们的名字。
- modules()方法：返回一个生成器，包括模块下的所有各个层级的模块，包括模块本身。
- named_modules()方法：返回一个生成器，包括模块下的所有各个层级的模块以及它们的名字，包括模块本身。

其中chidren()方法和named_children()方法较多使用。modules()方法和named_modules()方法较少使用，其功能可以通过多个named_children()的嵌套使用实现。

下面看一个例子：

```
class Net(nn.Module):

    def __init__(self):
        super(Net, self).__init__()

        self.embedding = nn.Embedding(num_embeddings =
        self.conv = nn.Sequential()
        self.conv.add_module("conv_1",nn.Conv1d(in_cha
        self.conv.add_module("pool_1",nn.MaxPool1d(ker
        self.conv.add_module("relu_1",nn.ReLU())
        self.conv.add_module("conv_2",nn.Conv1d(in_cha
        self.conv.add_module("pool_2",nn.MaxPool1d(ker
        self.conv.add_module("relu_2",nn.ReLU())
```

上面建立了一个神经网络，包含两个子模块，分别用的模型容器建立的（这个下面就会说）

我们这里重点是看看这个网络的子模块到底怎么访问到：

```
i = 0
for child in net.children():
    i+=1
    print(child,"\n")
print("child number",i)
```

结果如下：

```
Embedding(10000, 3, padding_idx=1)

Sequential(
  (conv_1): Conv1d(3, 16, kernel_size=(5,), stride=(1,))
  (pool_1): MaxPool1d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (relu_1): ReLU()
  (conv_2): Conv1d(16, 128, kernel_size=(2,), stride=(1,))
  (pool_2): MaxPool1d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (relu_2): ReLU()
)

Sequential(
  (flatten): Flatten()
  (linear): Linear(in_features=6144, out_features=1, bias=True)
  (sigmoid): Sigmoid()
)

child number 3
```

如果是net.named_children(),这个会在模块前面加上conv: ， dense:， 表示它们的名字。

下面我们通过named_children方法找到embedding层，并将其参数设置为不可训练(相当于冻结embedding层)。这个在迁移学习的时候非常有用。

```
children_dict = {name:module for name,module in net.nar

print(children_dict)
embedding = children_dict["embedding"]
embedding.requires_grad_(False) #冻结其参数


#可以看到其第一层embedding的参数已经不可以被训练了。
for param in embedding.parameters():
    print(param.requires_grad)     # False
    print(param.numel())    # 30000
```

关于Pytorch提取神经网络层结构、层参数及自定义初始化, 参考这篇文章
https://blog.csdn.net/weixin_41560402/article/details/108122910

3.4-3. 模型容器CONTAINERS

- 上面我们学习的模型的搭建过程，包括两个要素：构建子模块和拼接子模块，
- 在搭建模型中，还有一个非常重要的概念，那就是模型容器Containers。
- 下面我们就来看看模型容器Containers。依然是先观察整体框架：



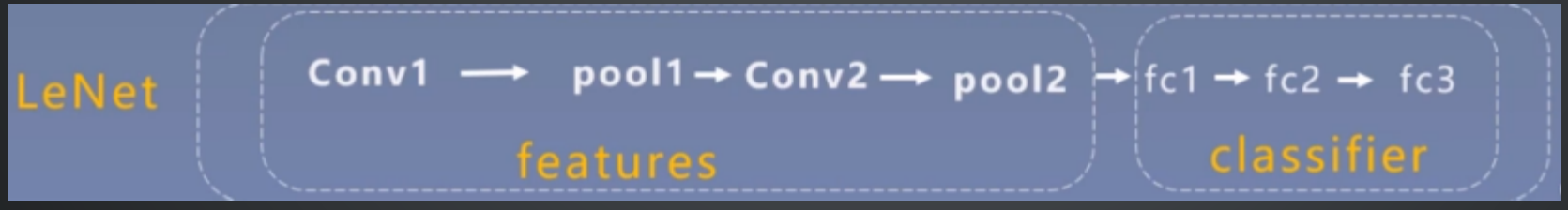
Containers这个容器里面包含3个子模块，分别是`nn.Sequential`, `nn.ModuleList`, `nn.ModuleDict`, 下面我们一一来看一看：

3.4-3.1 NN.SEQUENTIAL

这是nn.module的容器，用于**按顺序**包装一组网络层。

- 深度学习中更偏向于让网络自己提取特征，然后进行分类或者回归任务
- 所以就像上面的LeNet那样，对于图像的特征，我们完全不需要人为的设计， 只需要从前面加上卷积层让网络自己学习提取
- 后面加上几个全连接层进行分类等任务。
- 所以在深度学习时代习惯以全连接层为界限，将网络模型划分为特征提取模块和分类模块以便更好的管理网络。

所以我们的LeNet模型，可以把前面的那部分划分为特征提取部分，后面的全连接层为模型部分。



下面我们通过代码来观察，使用sequential包装一个LeNet，看看是怎么做的：

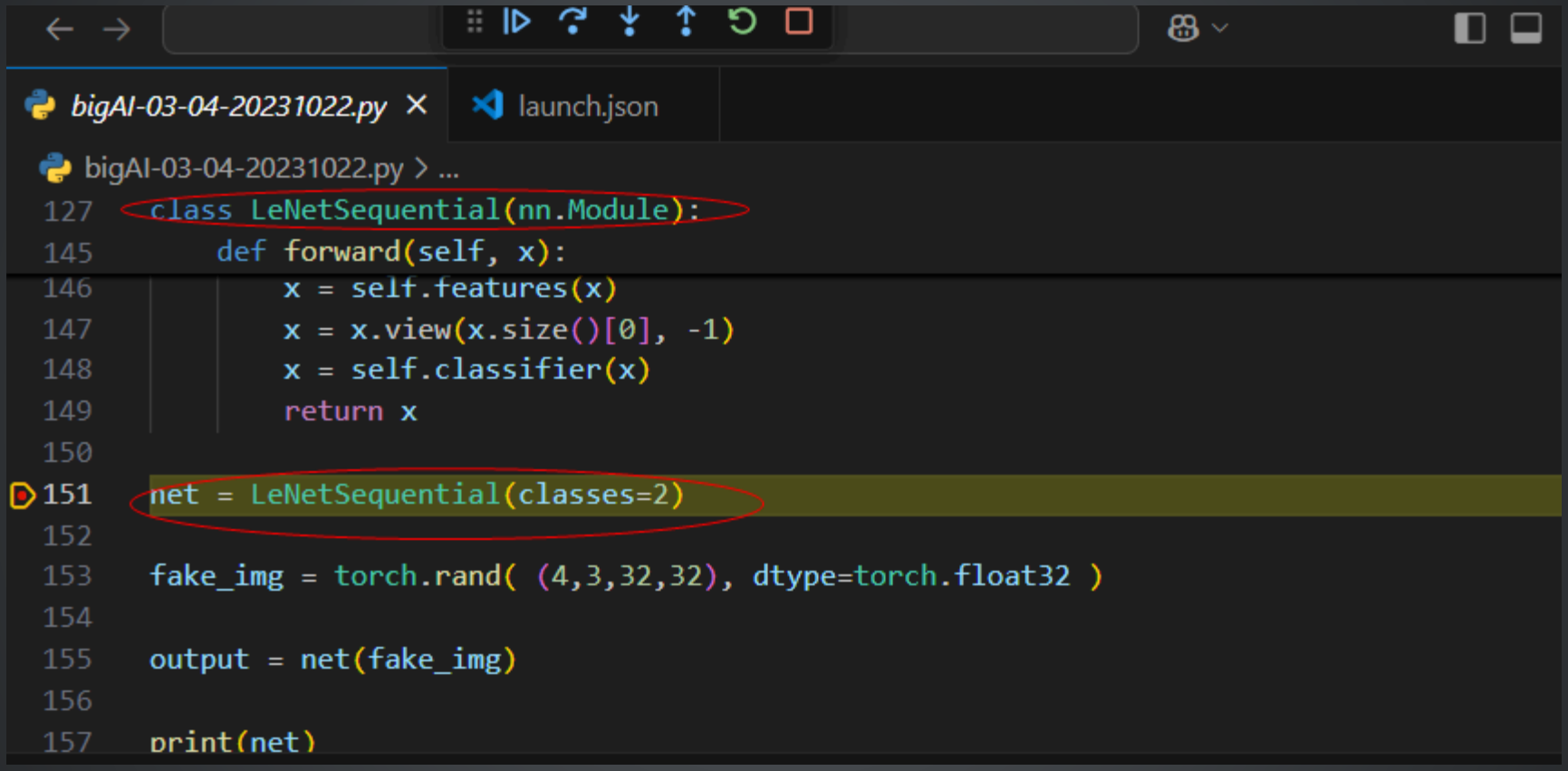
```
class LeNetSequential(nn.Module):
    def __init__(self, classes):
        super(LeNetSequential, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 6, 5),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
            nn.Conv2d(6, 16, 5),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),

        self.classifier = nn.Sequential(
            nn.Linear(16*5*5, 120),
```

可以看到，我们的LeNet在这里分成了两大部分

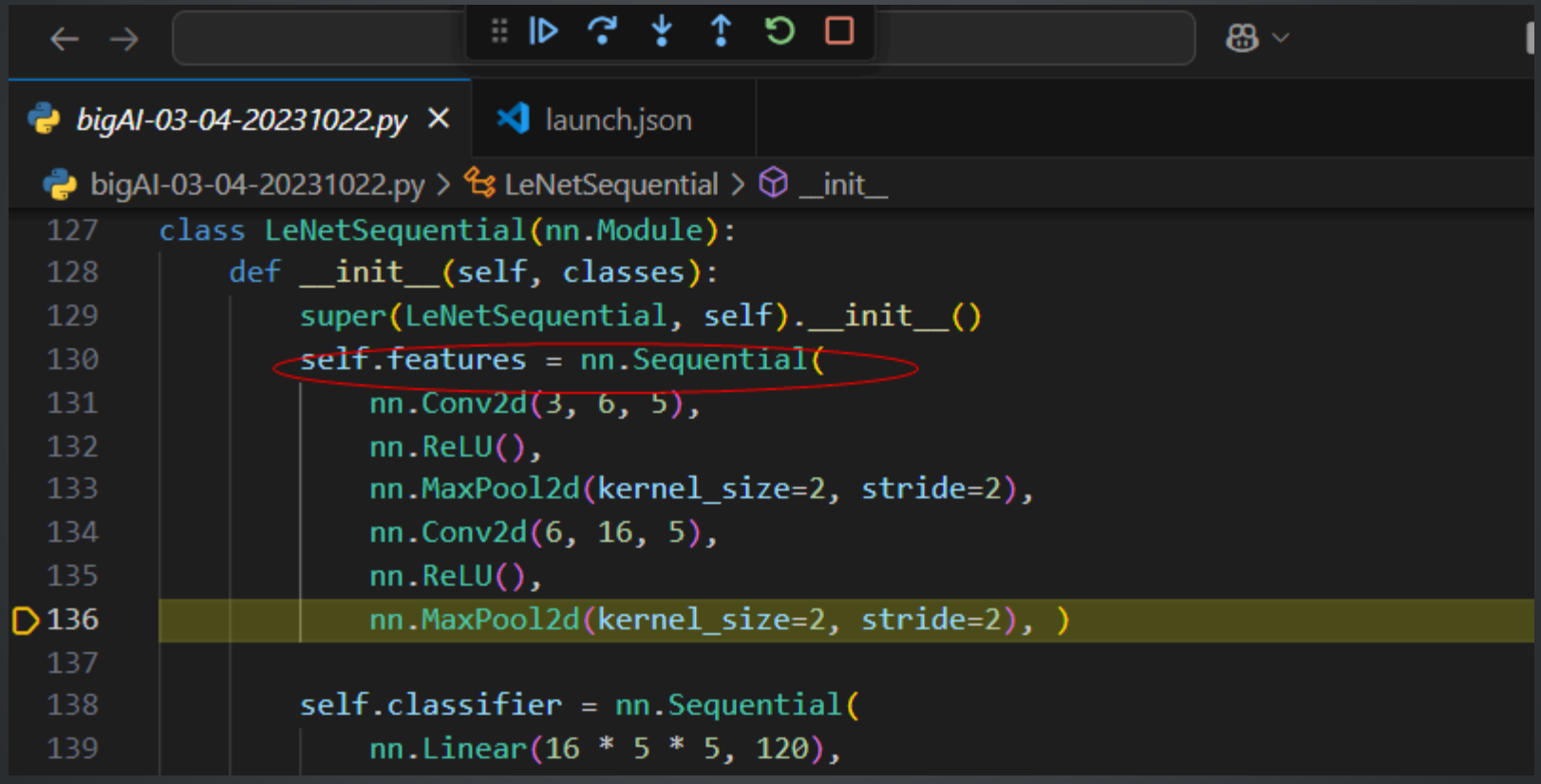
- 第一部分是features模块，用于特征提取，
- 第二部分是classifier部分，用于分类。
- 每一部分都是各种网络的堆叠，然后用sequential包装起来。
- 然后它的forward函数也比较简单，
    - 只需要features处理输入
    - 然后形状变换
    - 然后classifier就搞定。

下面通过代码调试，观察通过Sequential搭建的LeNet里面的一些属性，并且看看Sequential是一个什么样的机制：

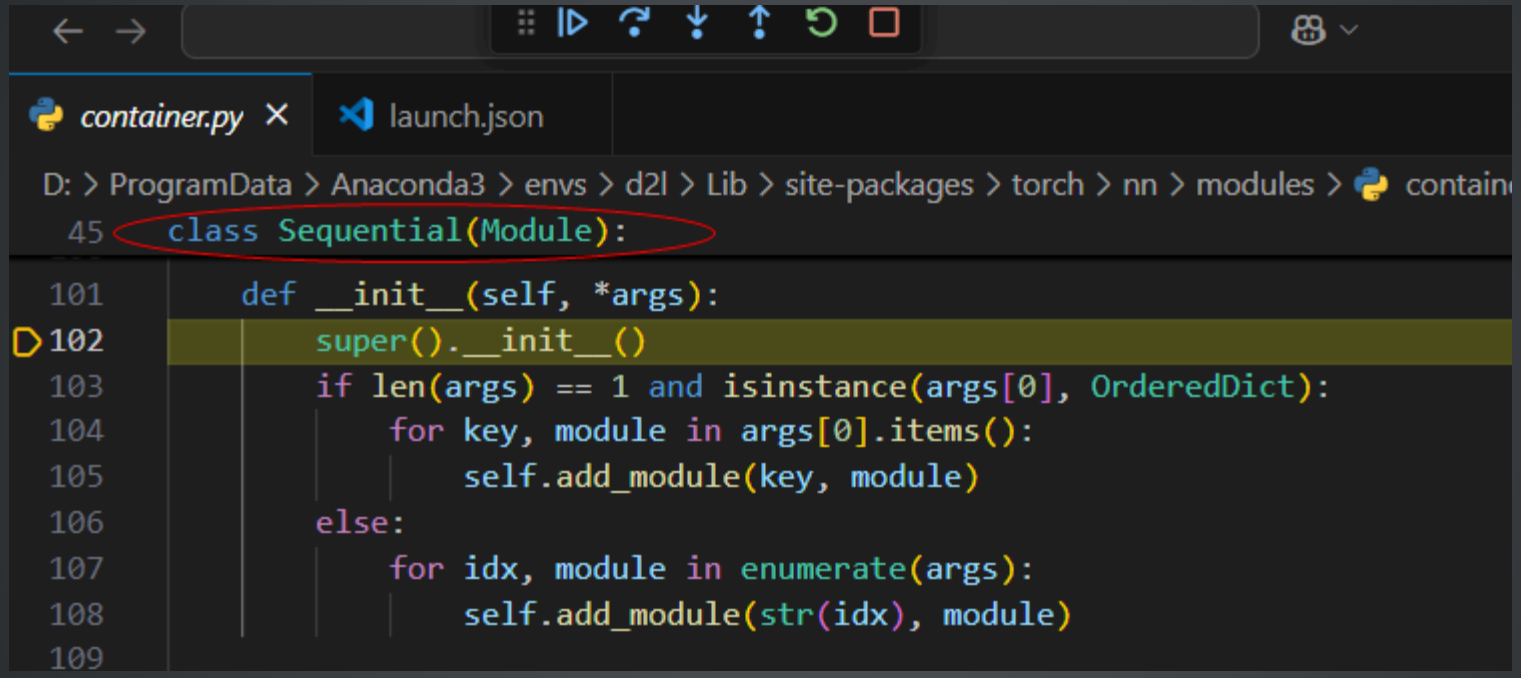


- 打断点，debug， 步入即可，这样会到了LeNetSequential这个类里面去
- 我们通过super进行初始化，因为这个继承的也是nn.Module，所以肯定也是若干的属性字典，这个就不管了，

stepover一步步的往下，到一个Sequential完成的位置停下来



- 然后，stepinto->stepout->stepinto, 进入container.py的Sequential这个类。
- 会发现`class Sequential(Module):`， 这说明Sequential也是继承于Module这个类的，所以它也会有那若干个参数字典。



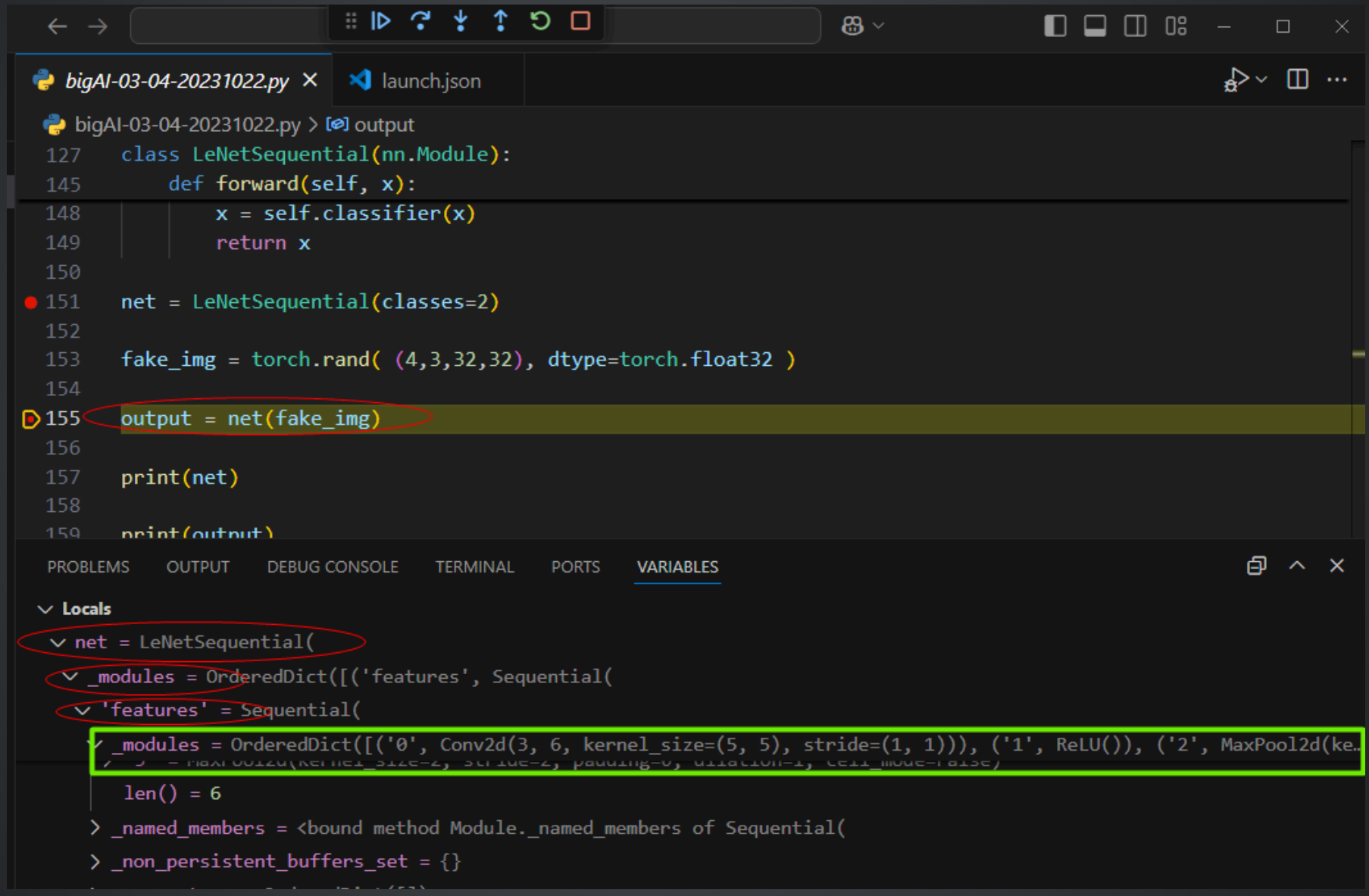
- 这样，一直stepover， 直到第5个子模块完
- 这样一个Sequential就建完了。
- 我们stepout回到主程序，然后往下执行，把第一个Sequential构建完毕。

下面的那个Sequential构建其实原理和第一个的一样了，所以不再进行调试查看

简单梳理一下Sequential的构建机制， 这个依然是继承Module类

- 也是在__init__方法中先调用父类去初始化若干有序字典，
- 然后再__init__里面完成各个子模块的参数存储。
- 这样，子模块构建完成

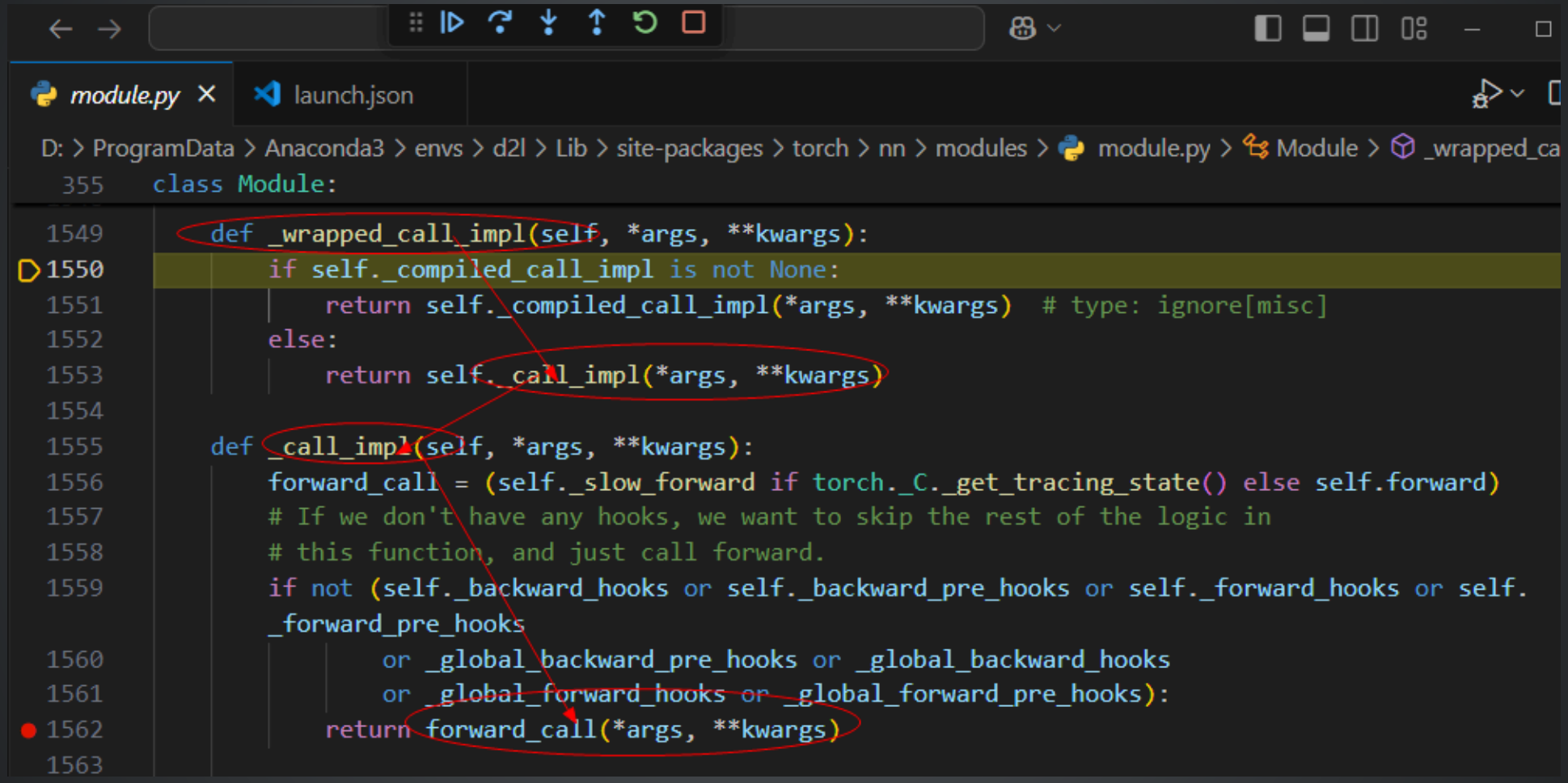
- 前向传播之前，神经网络构架已经是完整的，如下图



接下来，就是拼接子模块

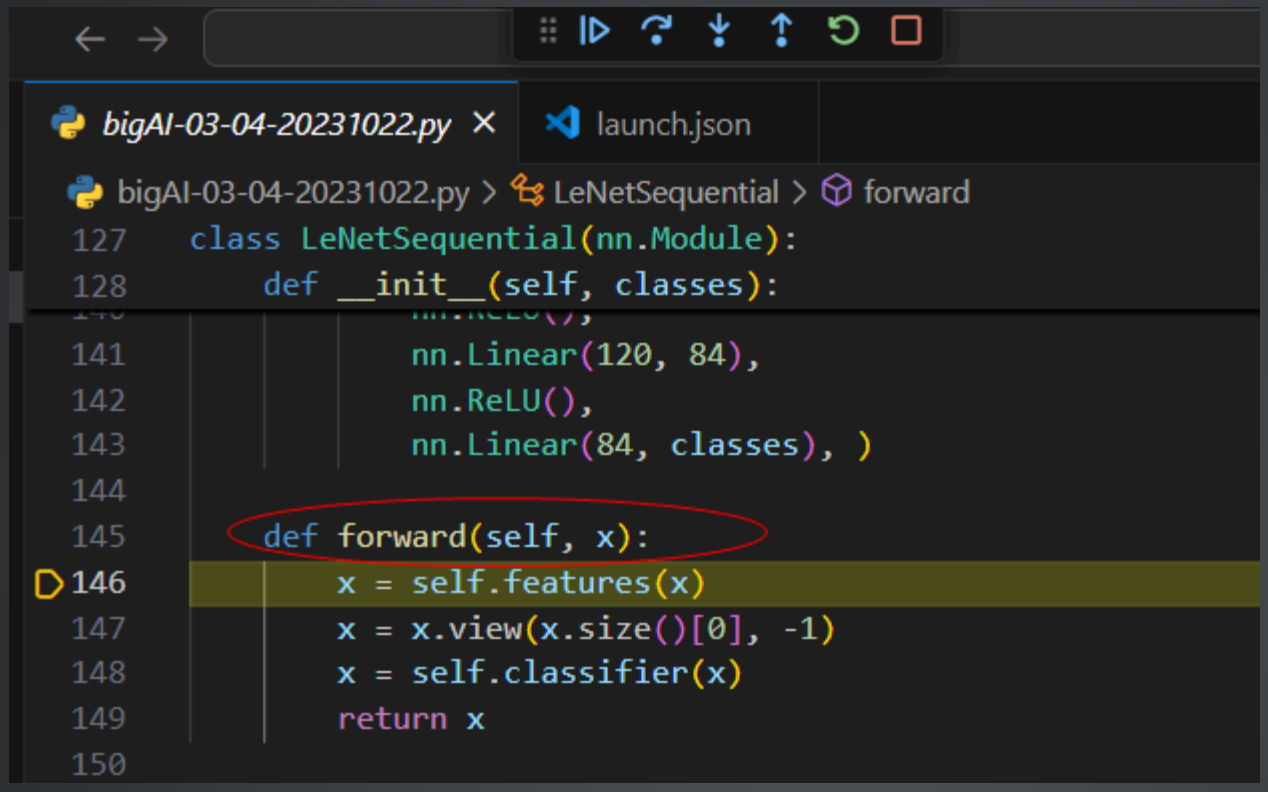
- 这个是通过前向传播函数完成的
- 下面我们看看Sequential是怎么进行拼接子模块的
- 我们看前向传播：

这时候步入这个net函数， 看看前向传播的实现过程，



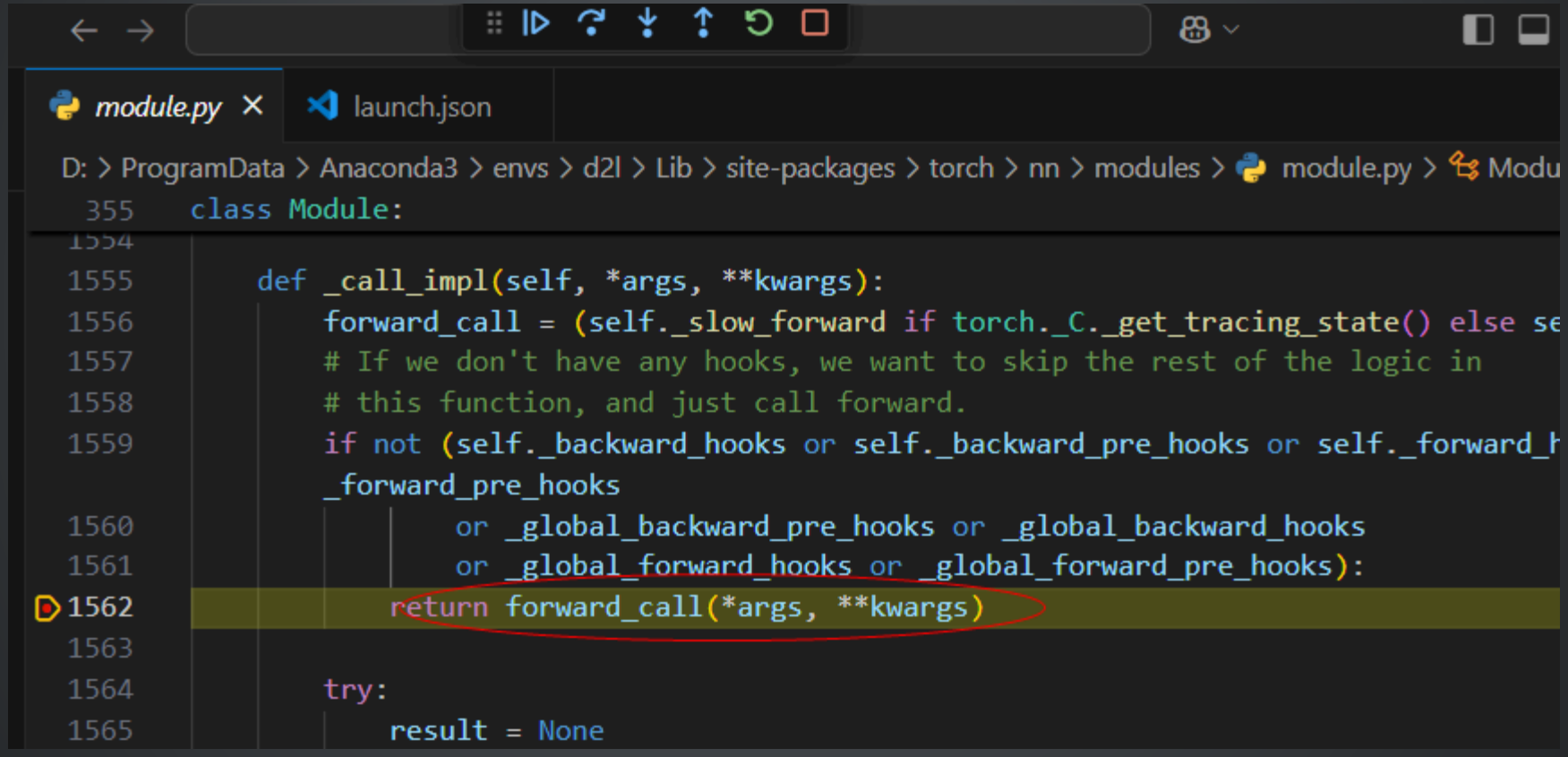
步入之后，就到了module.py的forward_call函数, 就是在这里面调用前向传播的：

- 步入之后，我们跳到了LeNetSequential类的前向传播函数



- 我们可以发现，完成第一个Sequential，也就是features的前向传播，只用了一句代码 `x = self.features(x)`，
- 这句代码竟然实现了6个子模块的前向传播，这是怎么做到的呢？ 在这行代码步入一探究竟：

- 由于self.features是一个Sequential，
- 而Sequential也是继承于Module，所以我们步入之后，依然是会跳到module.py的forward_call函数，



- 我们还是stepout到前向传播的那一行，然后步入看看Sequential的前向传播。

container.py × launch.json

D: > ProgramData > Anaconda3 > envs > d2l > Lib > site-packages > torch > nn > modules > container.py > Sequential > forward

```
45  class Sequential(Module):
213     # NB: We can't really type check this function as the type of input
214     # may change dynamically (as is tested in
215     # TestScript.test_sequential_intermediary_types).  Cannot annotate
216     # with Any as TorchScript expects a more precise type
217     def forward(self, input):
218         for module in self:
219             input = module(input)
220         return input
221
222     def append(self, module: Module) -> 'Sequential':
223         r"""Append a given module to the end.
224
225         Args:
226             module (nn.Module): module to append
227         """
228         self.add_module(str(len(self)), module)
229         return self
230
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS VARIABLES

Locals

```
self = Sequential(
  modules = OrderedDict([('0', Conv2d(3, 6, kernel_size=(5, 5), stride=(1, 1))), ('1', ReLU()), ('2', MaxPool2d(ke
  > special variables
  > function variables
  > '0' = Conv2d(3, 6, kernel_size=(5, 5), stride=(1, 1))
  > '1' = ReLU()
  > '2' = MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  > '3' = Conv2d(6, 16, kernel_size=(5, 5), stride=(1, 1))
  > '4' = ReLU()
  > '5' = MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  len() = 6
> named_members = <bound method Module._named_members of Sequential(
```

- 从上面可以看出，在Sequential的前向传播里面，遍历计算各层
- 即之前定义时候的那个_module那个有序的参数字典

- 这里面就是存的每个层的信息
- 前向传播的时候，就是遍历这个东西， 得到每个子模块，进行处理。
- 这里要注意一下，这是个串联机制，**也就是上一层的输出会是下一层的输入**。
- 所以要注意上下模型输入和输出的对应关系，数据格式，形状大小不要出错。

这就是Sequential的forward的机制运行的步骤了

SEQUENTIAL模型的简单小结一下

- 这一块的拼接完全是Sequential自己实现的，
- 在Sequential定义的时候，会把每一层的子模块的信息存入到它的_modules这个有序字典中，
- 然后前向传播的时候，==Sequential的forward函数就会遍历这个字典==，把每一层拿出来然后处理数据，
- 这是一个串行，上一层的输出正好是下一层的输入。这样通过这个迭代就可以完成前向传播过程。

```
158
159     print(output)
160
161     ##############################################
162     # 第二种Sequential的使用方法:
163
164     class LeNetSequentialOrderDict(nn.Module):
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS VARIABLES

```
LeNetSequential(
  (features): Sequential(
  (features): Sequential(
    (0): Conv2d(3, 6, kernel_size=(5, 5), stride=(1, 1))
    (1): ReLU()
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(6, 16, kernel_size=(5, 5), stride=(1, 1))
    (4): ReLU()
    (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (classifier): Sequential(
    (0): Linear(in_features=400, out_features=120, bias=True)
    (1): ReLU()
    (2): Linear(in_features=120, out_features=84, bias=True)
    (3): ReLU()
    (4): Linear(in_features=84, out_features=2, bias=True)
  )
)
```

- 拼接各层网络时候，网络层是有名字的，比如conv1， conv2

- 但是在Sequential里，每一层只有序号了，如果网络层成千上百的话，很难通过序号去索引网络层，
- 这时候，我们可以对网络层进行一个命名。

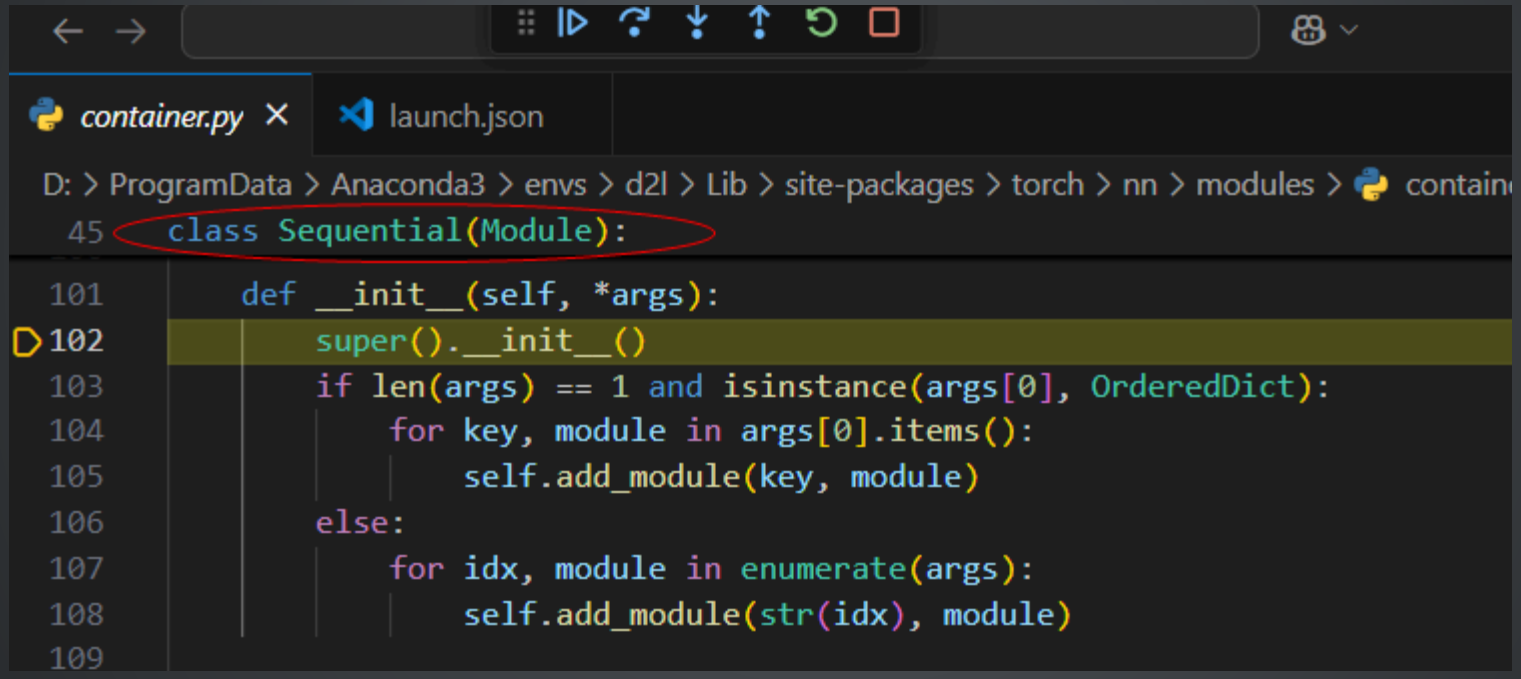
也就是第二种SEQUENTIAL的使用方法：

```
class LeNetSequentialOrderDict(nn.Module):
    def __init__(self, classes):
        super(LeNetSequentialOrderDict, self).__in

        self.features = nn.Sequential(OrderedDict(
            'conv1': nn.Conv2d(3, 6, 5),
            'relu1': nn.ReLU(inplace=True),
            'pool1': nn.MaxPool2d(kernel_size=2, s

            'conv2': nn.Conv2d(6, 16, 5),
            'relu2': nn.ReLU(inplace=True),
            'pool2': nn.MaxPool2d(kernel_size=2, s
        }))
```

- 这里面==Sequential包装的就是一个有序的字典， 字典中是网络名:网络层的形式==。
- 通过这个就可以对每一层网络进行命名，
- 那它是什么时候进行命名的呢？ 当然还是在定义的时候，
- 上面的某张图片里面我其实已经埋好了伏笔：



就是它了， 我们会看到， Sequential的初始化方法里面有个判断的

- if后面其实就是判断传入的是不是个有序字典，
- 我们上次Sequential里面直接是各个层，所以当时不满足if，跳到了下面的else，那里面是self.add_module(str(idx), module)这个很清楚了吧，就不是有序字典，说明我们没命名，那么就用数字索引命名，然后加入到_module有序参数字典中。
- 而这次我们是构建了个有序字典，那么就应该走if,这里面是 self.add_module(key, module)
    - 这一次我们命名了，所以就用我们的命名，把key(网络名):value(网络层)存入到_module有序参数字典中。
    - 这样，我们搭建的网络层就会有名字了。

下面对SEQUENTIAL进行一个总结：

- nn.Sequential是nn.module的容器， 用于**按顺序**包装一组网络层
    - 顺序性： 各网络层之间严格按照顺序构建，这时候一定要注意前后层数据的关系
    - 自带forward()：自带的forward里，通过for循环依次执行前向传播运算

提问

以下关于 `nn.Sequential` 的描述错误的是：

A. 它是一个按顺序包装网络层的容器。
B. 它会自动管理网络层的前向传播。
C. 它支持通过名称索引访问网络层。
D. 网络层之间必须严格按照顺序连接。

答案： C
解析：

- nn.Sequential 是一个按顺序包装网络层的容器，支持自动完成前向传播（选项 A 和 B 正确）。
- 网络层之间必须严格按照顺序连接（选项 D 正确）。
- 默认情况下，nn.Sequential 使用数字索引访问网络层，而非名称索引，除非使用 OrderedDict 显式命名（选项 C 错误）。

3.4-3.2 NN.MODULELIST

nn.ModuleList是nn.module的容器， 用于包装一组网络层， 以**迭代**方式调用网络层， 主要方法：

- `append()`: 在ModuleList后面添加网络层
- `extend()`: 拼接两个ModuleList
- `insert()`: 指定在ModuleList中位置插入网络层

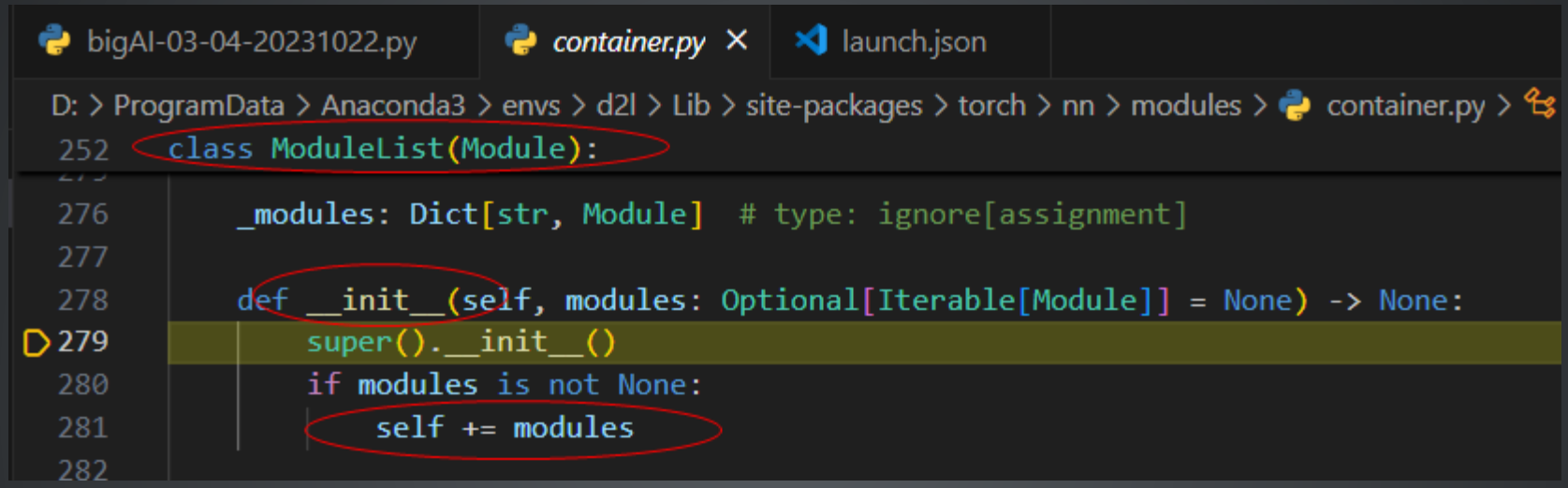
- 我们可以发现，这个方法的作用其实类似于我们的列表
- 只不过元素换成网络层而已，
- 下面我们学习ModuleList的使用，我们使用ModuleList来循环迭代的实现一个20个全连接层的网络的构建。

```
class ModuleList(nn.Module):
    def __init__(self):
        super(ModuleList, self).__init__()
        self.linears = nn.ModuleList([nn.Linear(10, 10

    def forward(self, x):
        for i, linear in enumerate(self.linears):
            x = linear(x)
        return x
```

- ModuleList构建网络层就可以使用列表生成式构建
- 然后前向传播的时候也是遍历每一层，进行计算即可。
- 我们下面调试就是看看这个ModuleList的初始化，是怎么把20个全连接层给连起来的。

```
194  ##############################################
195  # 我们使用ModuleList来循环迭代的实现一个20个全连接层的网络的构建。
196  class ModuleList(nn.Module):
197      def __init__(self):
198          super(ModuleList, self).__init__()
199          self.linears = nn.ModuleList([nn.Linear(10, 10) for i in range(20)])
200
201      def forward(self, x):
202          for i, linear in enumerate(self.linears):
203              x = linear(x)
204          return x
205
206  net = ModuleList()
207  fake_img = torch.randn((4, 10, 32, 32))
208  output = net(fake_img, 'conv', 'relu')  # 在这里可以选择我们的层进行组合
209  print(output)
```



- 可以看到这个modules是一个列表，里面就是这20个全连接层。
- 前向传播也比较简单了，用的for循环获取到每个网络层，这里就不调试了。
- 这样就完成了一个20层的全连接层的网络的实现。
- 借助nn.ModuleList只需要一行代码就可以搞定。这就是nn.ModuleList的使用。

3.4-3.3 NN.MODULEDICT

nn.ModuleDict是nn.module的容器， 用于包装一组网络层， 以**索引**方式调用网络层， 主要方法：

- `clear()`: 清空ModuleDict
- `items()`: 返回可迭代的键值对(key-value pairs)
- `keys()`: 返回字典的键(key)
- `values()`: 返回字典的值(value)
- `pop()`: 返回一对键值对， 并从字典中删除

可以通过ModuleDict实现网络层的选取，我们看下面的代码：

```
class ModuleDict(nn.Module):
    def __init__(self):
        super(ModuleDict, self).__init__()
        self.choices = nn.ModuleDict({
            'conv': nn.Conv2d(10, 10, 3),
            'pool': nn.MaxPool2d(3)
        })

        self.activations = nn.ModuleDict({
            'relu': nn.ReLU(),
            'prelu': nn.PReLU()
        })
```

- 这个理解起来， 前面通过`self.choices`这个ModuleDict可以选择卷积或者池化，
- 主程序：`output = net(fake_img, 'conv', 'relu')`，用于指定需要的网络层
- 对于激活函数的选择：类代码通过`self.activations`这个ModuleDict可以选取是用哪个激活函数， 这个东西在选择网络层的时候挺实用
    - 比如要做时间序列预测的时候，我们往往会用到GRU或者LSTM， 我们就可以通过这种方式来对比哪种网络的效果好。

提问

关于 nn.ModuleList 和 nn.ModuleDict 的区别，以下描述正确的是：

A. nn.ModuleList 支持通过键值对索引网络层，而 nn.ModuleDict 不支持。
B. 两者都可以用于存储网络层，但 nn.ModuleList 更适合迭代调用网络层。
C. 两者都不支持动态添加网络层。
D. 两者都必须按照顺序调用网络层。

答案： B
解析：

- nn.ModuleList 是一个容器，用于以列表形式存储网络层，适合通过迭代方式调用（选项 B 正确）。
- 而 nn.ModuleDict 是以字典形式存储网络层，支持通过键值对索引（选项 A 错误）。
- 两者都支持动态添加网络层，通过方法如 append() 或直接赋值（选项 C 错误）。
- 网络层调用顺序在 ModuleDict 中可以灵活指定，并非必须按顺序调用（选项 D 错误）。

到这里我们就学习了三个容器， nn.Sequential， nn.ModuleList， nn.ModuleDict。

下面总结一下它们的应用场合：

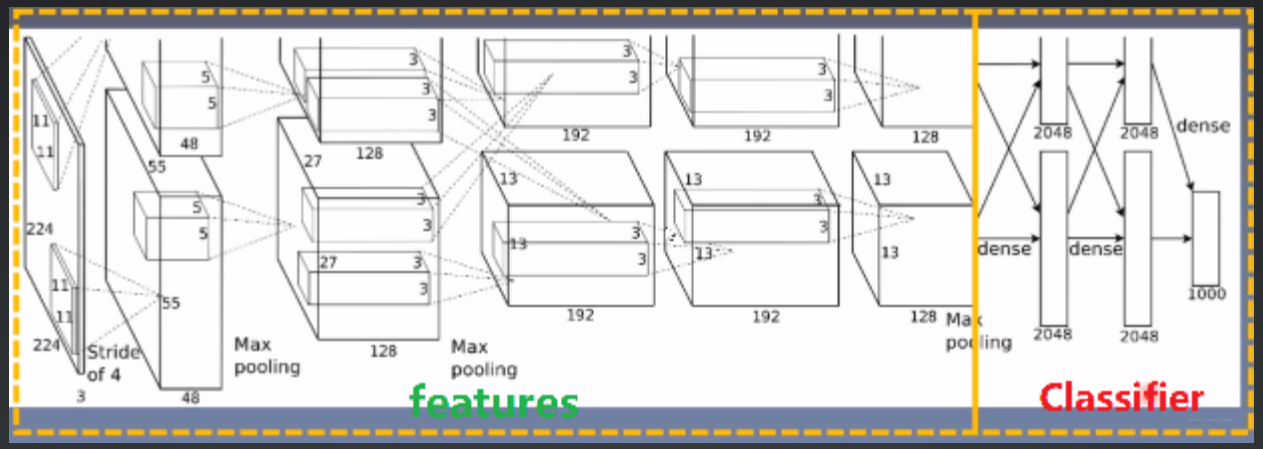
- nn.Sequential：顺序性，各网络层之间严格按顺序执行，常用于block构建
- nn.ModuleList：迭代性，常用于大量重复网构建，通过for循环实现重复构建
- nn.ModuleDict：索引性，常用于可选择的网络层

- 下面就来研究一个实际的网络模型，
- 这个是Pytorch提供的， 叫做AlexNet网络模型。

3.4-4. ALEXNET构建

这是一个划时代的卷积神经网络，2012年在ImageNet分类任务中获得了冠军，开创了卷积神经网络的新时代。 AlexNet的特点如下：

- 采用ReLu: 替换饱和激活函数， 减轻梯度消失
- 采用LRN(Local Response Normalization): 对数据归一化，减轻梯度消失（后面被Batch归一化取代了）
- Dropout： 提高全连接层的鲁棒性，增加网络的泛化能力
- Data Augmentation: TenCrop, 色彩修改



下面看看AlexNet的源代码：

```
class AlexNet(nn.Module):

    def __init__(self, num_classes=1000):
        super(AlexNet, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=11, stride=4,
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(64, 192, kernel_size=5, padding=
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
```

```
            nn.Conv2d(192, 384, kernel_size=3, padding=
            nn.ReLU(inplace=True)
```

bigAI-03-04-20231022.py | container.py | launch.json

bigAI-03-04-20231022.py > ...

```
237  ##############################################
238  # AlexNet的源代码:
239
240  class AlexNet(nn.Module):
241
242      def __init__(self, num_classes=1000):
243          super(AlexNet, self).__init__()
244          self.features = nn.Sequential(
245              nn.Conv2d(3, 64, kernel_size=11, stride=4, padding=2),
246              nn.ReLU(inplace=True),
247              nn.MaxPool2d(kernel_size=3, stride=2),
248              nn.Conv2d(64, 192, kernel_size=5, padding=2),
249              nn.ReLU(inplace=True),
250              nn.MaxPool2d(kernel_size=3, stride=2),
251              nn.Conv2d(192, 384, kernel_size=3, padding=1),
252              nn.ReLU(inplace=True),
253              nn.Conv2d(384, 256, kernel_size=3, padding=1),
254              nn.ReLU(inplace=True),
255              nn.Conv2d(256, 256, kernel_size=3, padding=1),
256              nn.ReLU(inplace=True),
257              nn.MaxPool2d(kernel_size=3, stride=2),
```

AlexNet就是用Sequential进行搭建的，分三部分，

- 第一部分是一个Sequential，由一系列的卷积池化模块构成，目的是提取图像的特征，
- 然后是一个全局的池化层把特征进行整合，
- 最后有一个Sequential是全连接层组成的，用于模型的分类。

这样就完成了AlexNet网络的搭建， forward函数汇总也是非常简单了，这里就不再详细赘述了。

到这里就基本上完成了这本节内容

- 如果你理解了这篇文章的内容，就会发现理解像AlexNet这样的网络构建非常简单，
- 当然在Pytorch的models模块中还有很多其他的经典网络，像googlenet， vgg，ResNet等很多，
- 学习了今天的知识之后，这些模型的构建都可以去查看了。
- 不管这些模型多么复杂， 构建依然还是我们的nn.Module， Sequential，ModuleList， ModuleDict的这些内容去构建的，所以我们依然可以看懂这些网络的逻辑。

3.4-5. 总结

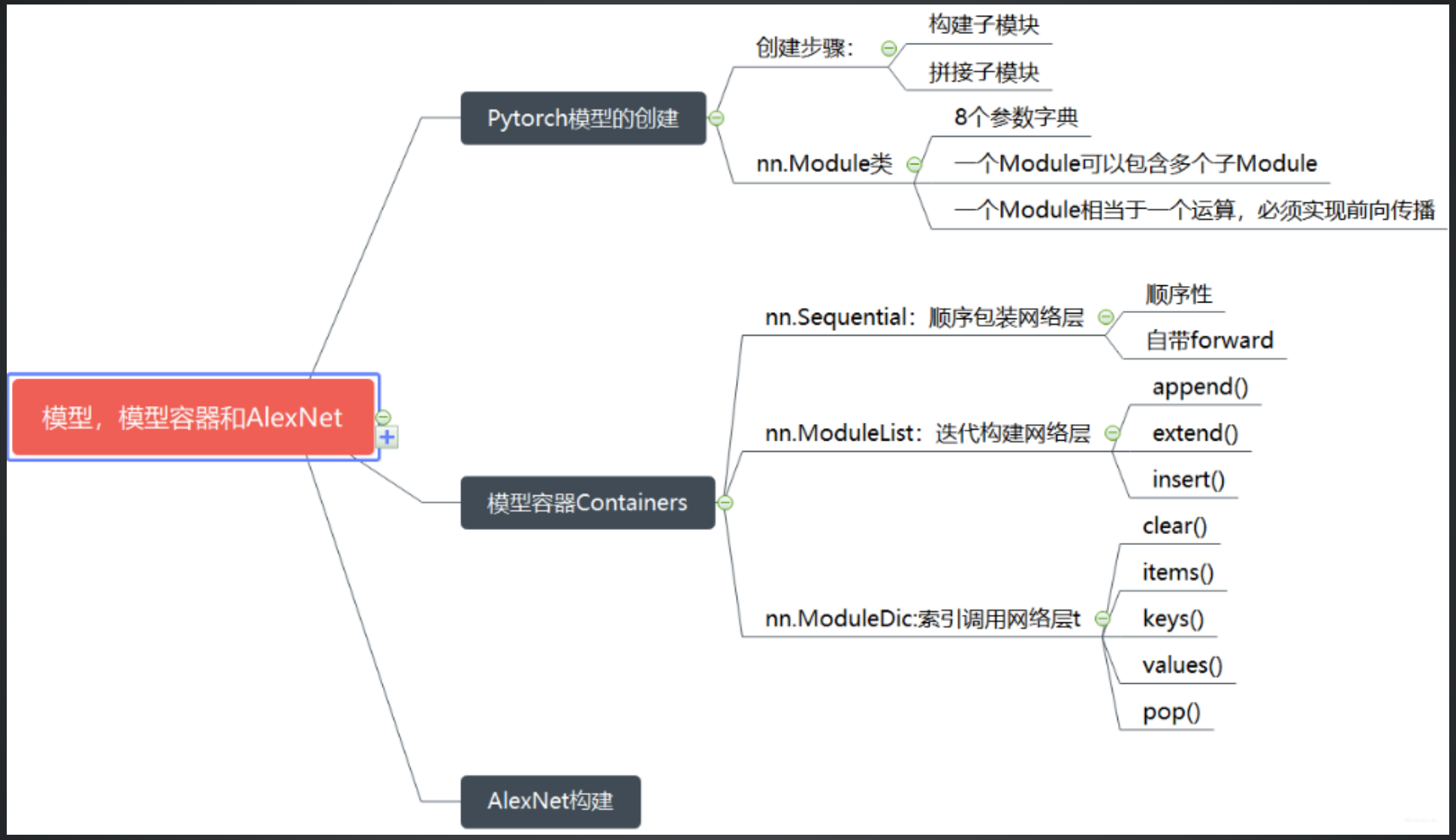
本节的内容有点多，并且还非常的重要，快速梳理总结为主要3大块

- 第一块就是Pytorch模型的构建步骤有两个子模块的构建和拼接，
  - 然后就是学习了非常重要的一个类叫做nn.Module，这个是非常重要的，后面的模型搭建中我们都得继承这个类，这就是祖宗级别的人物了。
  - 这里面有8个重要的参数字典，其中_parameters和_modules更是重中之重，所以以LeNet为例，通过代码调试的方式重点学习了LeNet的构建过程和细节部分。

- 第二块是我们的模型容器Containers部分
    - 这里面先学习了nn.Sequential， 这个是顺序搭建每个子模块， 常用于block构建，
    - 依然是通过代码调试看了它的初始化和自动前向传播机制。
    - 然后是nn.ModuleList， 这个类似于列表，常用于搭建结构相同的网络子模块， 特点就是可迭代。
    - 最后是nn.ModuleDict， 这个的特点是索引性，类似于我们的python字典，常用于可选择的网络层。

- 第三块就是根据上面的所学分析了一个非常经典的卷积神经网络AlexNet的构建过程，
    - 当然，只是可以去看懂网络结构的代码，
    - 如果想达到用Pytorch写神经网络的地步，就需要后面多多实践

下面依然是一张思维导图把知识连起来：



本节的知识就到这里，下一次我们就从这一次的基础上更进一步，学习几个比较重要的子模块，比如卷积，池化， 全连接等😉